LAPORAN
UJIAN TENGAH SEMESTER

PEMROGRAMAN BERORIENTASI OBJEK

Muhammad Rizky Novaldi
04231062

Dosen Pengampu :

Himawan Wicaksono, S.ST.,M.T.
NIP. 198904012021211002

PROGRAM STUDI TEKNIK ELEKTRO

JURUSAN TEKNIK ELEKTRO INFORMATIKA DAN BISNIS

FAKULTAS SAINS DAN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI KALIMANTAN

BALIKPAPAN

2026

TAHAP 1
Analisis SDLC & Manajemen Proyek

1.1 Alasan Sistem Lebih Aman Menggunakan Paradigma Berorientasi Objek (OOP)

Sistem Layanan Fotokopi Mandiri (ITK-Print) dinilai sangat sesuai untuk dikembangkan menggunakan paradigma Berorientasi Objek (OOP). Kesesuaian ini didasari oleh karakteristik sistem yang melibatkan entitas-entitas yang saling berinteraksi, yakni Pelanggan, MesinPrint, dan Dokumen. Melalui pendekatan OOP, masing-masing entitas tersebut dimodelkan sebagai objek independen yang menyimpan atribut sekaligus mengatur perilaku atau fungsinya secara mandiri. Pendekatan ini menjadikan struktur sistem lebih terorganisir serta pengelolaan data transaksi menjadi lebih terpusat.

Dari sisi keamanan, OOP jauh lebih unggul dibandingkan paradigma terstruktur (prosedural) karena terdapat dukungan mutlak terhadap konsep enkapsulasi (data hiding). Data-data yang bersifat sensitif, seperti jumlah saldo pelanggan, kapasitas tinta, dan ketersediaan stok kertas pada mesin cetak, dikunci secara tertutup agar tidak dapat diakses maupun dimodifikasi secara sembarangan dari luar kelas (class). Perubahan terhadap atribut tersebut hanya diizinkan melalui jalur resmi berupa method yang memuat aturan bisnis secara ketat. Sebagai contoh, saldo pelanggan hanya dapat dikurangi secara otomatis apabila logika sistem telah memvalidasi bahwa kuantitas tinta dan kertas dipastikan mencukupi untuk mencetak spesifikasi dokumen.

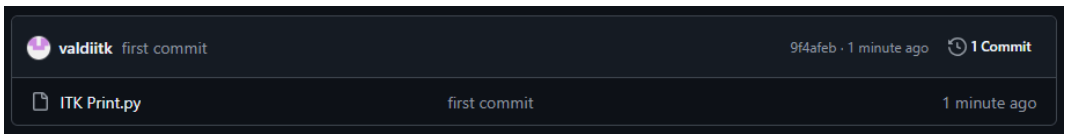
Apabila paradigma prosedural digunakan dan data operasional dibiarkan terbuka sebagai variabel global, sistem akan sangat rentan terhadap manipulasi atau anomali modifikasi data. Masalah fatal yang sangat mungkin terjadi adalah mesin cetak tetap merespons instruksi pencetakan meskipun atribut tinta atau kertas bernilai nol (0), atau proses pencetakan tetap diotorisasi kendati saldo pelanggan berada di bawah total nominal biaya (Jumlah Halaman × Harga Per Lembar). Tidak adanya pembatasan pada variabel global juga berpotensi memicu pengurangan saldo secara ilegal tanpa adanya transaksi pencetakan yang berhasil. Kesalahan struktural semacam ini akan merusak integritas sistem dan menimbulkan kerugian operasional yang berkelanjutan

1.2 Implementasi Git dan GitHub untuk Version Control

Proses manajemen proyek diawali dengan inisialisasi repositori lokal menggunakan perintah git init, yang kemudian dihubungkan ke repositori publik pada platform GitHub. Seluruh riwayat pengerjaan proyek didokumentasikan melalui sistem version control ini agar setiap perubahan kode dapat terlacak dengan rapi dan kronologis. Berdasarkan tangkapan layar yang dilampirkan, terdapat tiga commit utama yang mencerminkan tiga tahapan pengerjaan proyek secara berurutan.

```
anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git init
Reinitialized existing Git repository in C:/UTS PBO/.git/

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git add .

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git commit -m "first commit"
[master (root-commit) 9f4afeb] first commit
 1 file changed, 284 insertions(+)
 create mode 100644 ITK Print.py

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git remote add origin https://github.com/valdiitk/UTS-PBO.git

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 2.67 KiB | 1.33 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/valdiitk/UTS-PBO.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

Gambar 1. Monitor Git



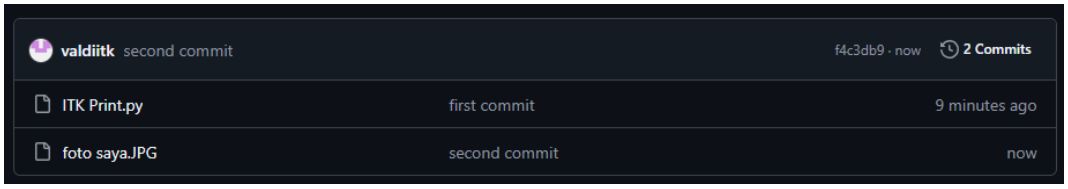
Gambar 2. Hasil Pada Github

Pada commit pertama dengan pesan “first commit”, file utama bernama ITK Print.py diunggah ke dalam repositori. Proses pengiriman data dilakukan menggunakan perintah git push -u origin master, yang berhasil mentransfer

sebanyak 2,67 KiB data ke server GitHub. Keberhasilan proses ini ditandai dengan munculnya branch master pada repositori dan telah terhubung secara sinkron dengan branch lokal.

```
anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git add .

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git commit -m "second commit"
[master f4c3db9] second commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 foto saya.JPG

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 596.27 KiB | 33.13 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/valdiitk/UTS-PBO.git
   9f4afeb..f4c3db9  master -> master
branch 'master' set up to track 'origin/master'.
```

Gambar 3. Tampilan Monitor Git Kedua



Gambar 4. Hasil Kedua Pada Github

Commit kedua dengan pesan “second commit” digunakan untuk menambahkan file foto saya.JPG sebagai pelengkap dokumentasi proyek. Pada tahapan ini, tercatat bahwa sebanyak 596,27 KiB data berhasil ditransfer, yang sebagian besar merupakan ukuran file gambar yang diunggah. Repositori pada GitHub kemudian memperlihatkan dua file sekaligus, yakni ITK Print.py dari commit pertama dan foto saya.JPG dari commit kedua.

```
anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git add .

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git commit -m "third commit"
[master 60551c1] third commit
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 UTS PBO.pdf

anggi@DESKTOP-NNJ70N5 MINGW64 /c/UTS PBO (master)
$ git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 340.81 KiB | 37.87 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/valdiitk/UTS-PBO.git
   f4c3db9..60551c1  master -> master
branch 'master' set up to track 'origin/master'.
```

Gambar 5. Tampilan Monitor Git Ketiga

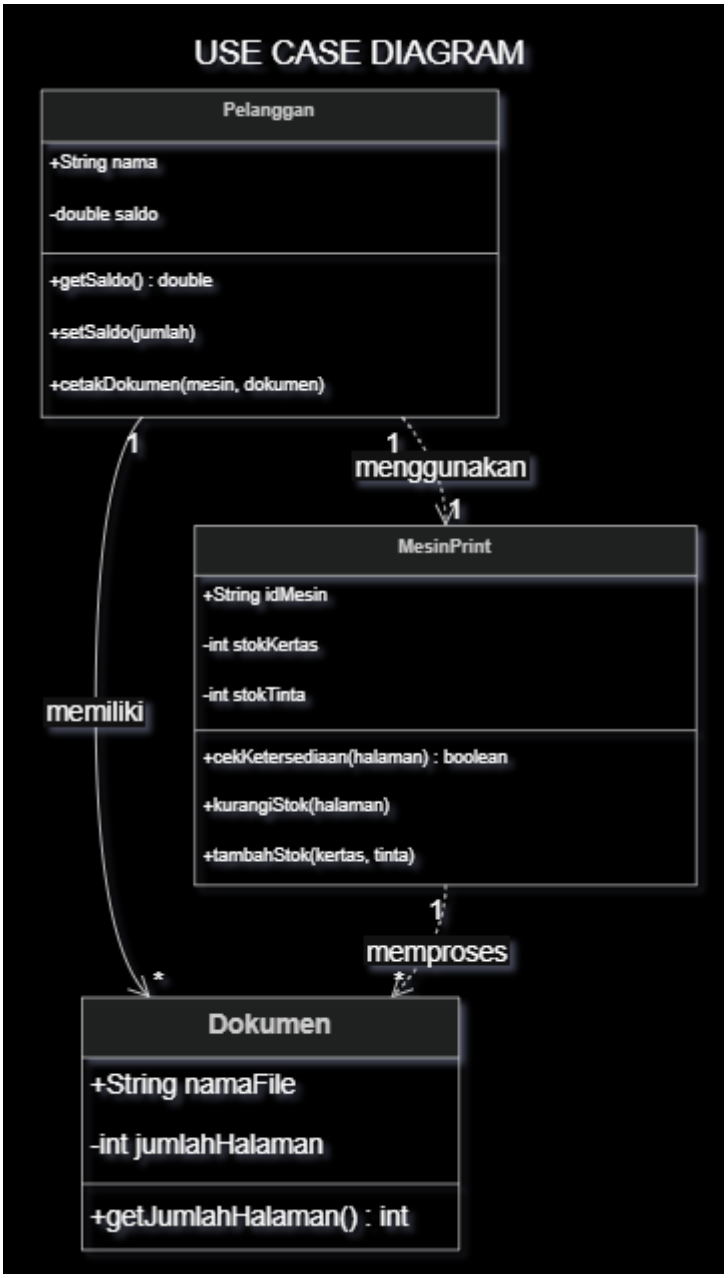
Gambar 6. Hasil Ketiga Pada Github

Commit ketiga dengan pesan “third commit” merupakan tahap final pengiriman data, yaitu penambahan file laporan UTS PBO.pdf ke dalam repositori. Pada commit ini, data sebesar 340,81 KiB berhasil dikirimkan ke server GitHub. Hasil akhir pada antarmuka repositori GitHub menampilkan keseluruhan tiga file yang terdaftar sesuai dengan riwayat commit masing-masing, menandakan bahwa seluruh komponen proyek telah terunggah dengan lengkap dan terstruktur.

TAHAP 2

Pemodelan Sistem

2.1 Perancangan *Use Case Diagram*



Gambar 7. *Use Case Diagram*

Use case diagram pada Sistem Layanan Fotokopi Mandiri (ITK-Print) dirancang untuk menggambarkan interaksi antara aktor-aktor utama dengan fungsionalitas yang tersedia dalam sistem. Berdasarkan diagram yang telah dibuat,

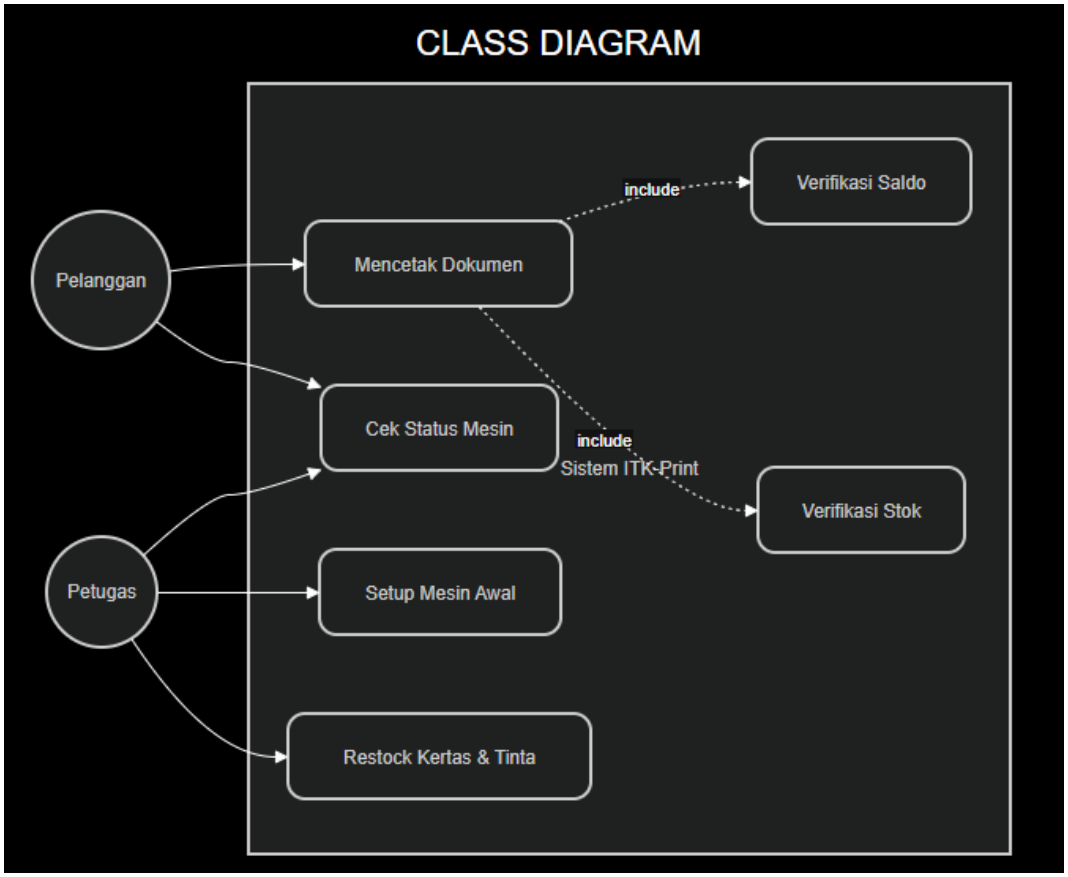
terdapat tiga entitas utama yang saling terhubung, yaitu Pelanggan, MesinPrint, dan Dokumen.

Pelanggan merupakan aktor primer yang berinteraksi langsung dengan sistem. Atribut yang dimiliki oleh kelas Pelanggan terdiri atas nama (bertipe String dengan akses publik) dan saldo (bertipe double dengan akses privat). Adapun method yang tersedia pada kelas ini mencakup getSaldo() yang mengembalikan nilai bertipe double, setSaldo(jumlah) untuk memperbarui saldo secara terkontrol, serta cetakDokumen(mesin, dokumen) yang merupakan method utama untuk memproses permintaan pencetakan. Relasi yang terbentuk antara Pelanggan dan MesinPrint adalah relasi “menggunakan” dengan kardinalitas 1 terhadap 1, yang menandakan bahwa seorang pelanggan menggunakan tepat satu mesin dalam satu transaksi.

MesinPrint berperan sebagai entitas yang mengelola sumber daya fisik pencetakan. Kelas ini memiliki atribut idMesin (String, publik), stokKertas (int, privat), dan stokTinta (int, privat). Method yang disediakan mencakup cekKetersediaan(halaman) yang mengembalikan nilai boolean, kurangiStok(halaman) untuk memotong stok secara proporsional, serta tambahStok(kertas, tinta) untuk proses pengisian ulang. MesinPrint dihubungkan dengan Dokumen melalui relasi “memproses” dengan kardinalitas 1 terhadap banyak (*), yang berarti satu mesin dapat memproses lebih dari satu dokumen.

Dokumen merupakan entitas yang merepresentasikan objek yang akan dicetak. Kelas ini memiliki atribut namaFile (String, publik) dan jumlahHalaman (int, privat), serta satu method yaitu getJumlahHalaman() yang bersifat publik dan mengembalikan nilai integer. Relasi antara Pelanggan dan Dokumen bersifat “memiliki” dengan kardinalitas 1 terhadap banyak (*), yang mencerminkan bahwa satu pelanggan dapat memiliki lebih dari satu dokumen untuk dicetak. Keseluruhan relasi antar entitas ini membentuk alur kerja sistem yang terstruktur, mulai dari permintaan pelanggan, pemrosesan oleh mesin, hingga pengeluaran dokumen hasil cetakan.

2.2 Perancangan *Class Diagram*



Gambar 8. *Class Diagram*

Class diagram Sistem Layanan Fotokopi Mandiri (ITK-Print) dibangun menggunakan struktur kelas yang saling terintegrasi untuk mengelola data operasional secara sistematis. Diagram ini terdiri atas dua aktor, yaitu Pelanggan dan Petugas, yang masing-masing memiliki hak akses terhadap fungsi-fungsi tertentu dalam sistem.

Dari sisi Pelanggan, terdapat dua use case utama yang dapat diakses, yaitu Mencetak Dokumen dan Cek Status Mesin. Proses Mencetak Dokumen memiliki ketergantungan wajib (relasi include) terhadap fungsi Verifikasi Saldo, yang berarti setiap kali pelanggan hendak mencetak dokumen, sistem secara otomatis memeriksa kecukupan saldo terlebih dahulu. Demikian pula, proses Cek Status Mesin secara otomatis menyertakan fungsi Verifikasi Stok melalui relasi include terhadap Sistem ITK-Print sebagai inti pemrosesan.

Dari sisi Petugas, terdapat dua use case yang menjadi tanggung jawab operasional, yaitu Setup Mesin Awal dan Restock Kertas & Tinta. Setup Mesin

Awal dilakukan pada awal sesi operasional untuk mendefinisikan identitas mesin serta stok kertas dan tinta awal. Sementara itu, Restock Kertas & Tinta merupakan fungsi yang dijalankan ketika salah satu atau kedua sumber daya fisik pada mesin telah habis atau hampir habis. Pembagian hak akses antara Pelanggan dan Petugas ini mencerminkan pemisahan tanggung jawab yang jelas dalam sistem, sehingga masing-masing aktor hanya dapat mengakses fungsi yang sesuai dengan perannya.

TAHAP 3

Implementasi Enkapsulasi Kotlin

3.1 *Source Code*

Setelah memahami secara teori bahwa paradigma Berorientasi Objek (OOP) memberikan keamanan lebih melalui enkapsulasi, langkah selanjutnya adalah menerapkannya ke dalam kode program. Bahasa pemrograman yang digunakan adalah Python. Berikut adalah source code lengkap yang digunakan:

Tabel 1. *Source Code*

Source Code

```
class Dokumen:
    def __init__(self, nama_file: str, jumlah_halaman: int):
        self.nama_file = nama_file
        self.__jumlah_halaman = jumlah_halaman

    def get_jumlah_halaman(self) -> int:
        return self.__jumlah_halaman

class MesinPrint:
    KAPASITAS_MAX_KERTAS = 100
    KAPASITAS_MAX_TINTA  = 100
    BATAS_PERINGATAN     = 20

    def __init__(self, id_mesin: str, stok_kertas: int, stok_tinta: int):
        self.id_mesin      = id_mesin
        self.__stok_kertas = stok_kertas
        self.__stok_tinta  = stok_tinta
        self.__total_cetak = 0

    def get_stok_kertas(self)  -> int: return self.__stok_kertas
    def get_stok_tinta(self)   -> int: return self.__stok_tinta
    def get_total_cetak(self)  -> int: return self.__total_cetak

    def cek_mesin_siap(self) -> bool:
        return self.__stok_kertas > 0 and self.__stok_tinta > 0

    def stok_habis(self) -> bool:
        return self.__stok_kertas == 0 or self.__stok_tinta == 0

    def stok_cukup_untuk(self, halaman: int) -> bool:
        return self.__stok_kertas >= halaman and self.__stok_tinta >= halaman

    def cek_peringatan(self):
        if 0 < self.__stok_kertas <= self.BATAS_PERINGATAN:
            print(f"  *** Kertas hampir habis! Sisa {self.__stok_kertas} lembar.")
```

```
        if 0 < self.__stok_tinta <= self.BATAS_PERINGATAN:
            print(f"  *** Tinta hampir habis! Sisa {self.__stok_tinta} unit.")

    def kurangi_stok(self, halaman: int):
        self.__stok_kertas -= halaman
        self.__stok_tinta  -= halaman
        self.__total_cetak += halaman

    def restock(self, tambah_kertas: int = 0, tambah_tinta: int = 0):
        print("\n" + "="*55)
        print(f"  RESTOCK MESIN {self.id_mesin}")
        print("-"*55)

        if tambah_kertas > 0:
            sebelum = self.__stok_kertas
            maks    = self.KAPASITAS_MAX_KERTAS
            if sebelum >= maks:
                print(f"  Kertas sudah penuh ({sebelum}/{maks}).")
            else:
                aktual = min(tambah_kertas, maks - sebelum)
                self.__stok_kertas += aktual
                print(f"  Kertas : {sebelum} + {aktual} = {self.__stok_kertas} lembar")
                if aktual < tambah_kertas:
                    print(f"            (dibatasi kapasitas maks {maks})")

        if tambah_tinta > 0:
            sebelum = self.__stok_tinta
            maks    = self.KAPASITAS_MAX_TINTA
            if sebelum >= maks:
                print(f"  Tinta  sudah penuh ({sebelum}/{maks}).")
            else:
                aktual = min(tambah_tinta, maks - sebelum)
                self.__stok_tinta += aktual
                print(f"  Tinta  : {sebelum} + {aktual} = {self.__stok_tinta} unit")
                if aktual < tambah_tinta:
                    print(f"            (dibatasi kapasitas maks {maks})")

        print("="*55)

    def status_bar(self) -> str:
                                bk          =          self.__buat_bar(self.__stok_kertas,
self.KAPASITAS_MAX_KERTAS)
        bt = self.__buat_bar(self.__stok_tinta,  self.KAPASITAS_MAX_TINTA)
        return (
            f"  Mesin       : {self.id_mesin}\n"
                                        f"         Kertas                        :        {bk}
{self.__stok_kertas}/{self.KAPASITAS_MAX_KERTAS}\n"
```

```
                                        f"        Tinta                          :        {bt}
{self.__stok_tinta}/{self.KAPASITAS_MAX_TINTA}\n"
            f"  Total cetak : {self.__total_cetak} halaman"
        )

    def __buat_bar(self, nilai: int, maks: int, panjang: int = 15) -> str:
        isi = round((nilai / maks) * panjang) if maks > 0 else 0
        return "[" + "#" * isi + "." * (panjang - isi) + "]"

class Pelanggan:
    def __init__(self, nama: str, saldo: float):
        self.nama    = nama
        self.__saldo = saldo

    def get_saldo(self) -> float:
        return self.__saldo

    def potong_saldo(self, jumlah: float):
        self.__saldo -= jumlah

    def cetak_dokumen(self, dokumen: Dokumen, mesin: MesinPrint,
                      harga_per_lembar: float) -> str:
        halaman     = dokumen.get_jumlah_halaman()
        total_biaya = halaman * harga_per_lembar

        print("\n" + "="*55)
        print(f"  TRANSAKSI -- {self.nama}")
        print("-"*55)
        print(f"  File      : {dokumen.nama_file}")
            print(f"   Halaman      :  {halaman}  x  Rp{harga_per_lembar:,.0f}  =
Rp{total_biaya:,.0f}")
        print(f"  Saldo     : Rp{self.__saldo:,.0f}")
            print(f"   Kertas         :  {mesin.get_stok_kertas()}  lembar  |  Tinta:
{mesin.get_stok_tinta()} unit")
        print("-"*55)

        if not mesin.stok_cukup_untuk(halaman):
            kertas_kurang = mesin.get_stok_kertas() < halaman
            tinta_kurang  = mesin.get_stok_tinta()  < halaman
            if kertas_kurang and tinta_kurang:
                print(f"  GAGAL: Stok kertas ({mesin.get_stok_kertas()}) DAN "
                      f"tinta ({mesin.get_stok_tinta()}) tidak cukup untuk {halaman}
halaman!")
            elif kertas_kurang:
                print(f"  GAGAL: Stok kertas tidak cukup! "
                      f"Butuh {halaman}, tersisa {mesin.get_stok_kertas()}.")
            else:
                print(f"  GAGAL: Stok tinta tidak cukup! "
```

```
                f"Butuh {halaman}, tersisa {mesin.get_stok_tinta()}.")
            print("="*55)
            return 'stok_habis'

        if self.__saldo < total_biaya:
            kekurangan = total_biaya - self.__saldo
            print(f"  GAGAL: Saldo tidak cukup!")
            print(f"        Dibutuhkan : Rp{total_biaya:,.0f}")
            print(f"        Saldo Anda : Rp{self.__saldo:,.0f}")
            print(f"        Kurang     : Rp{kekurangan:,.0f}")
            print("="*55)
            return 'saldo_kurang'

        self.potong_saldo(total_biaya)
        mesin.kurangi_stok(halaman)
        print(f"  SUKSES: Dokumen dicetak!")
        print(f"  Saldo terpotong : Rp{total_biaya:,.0f}")
        print(f"  Saldo tersisa   : Rp{self.__saldo:,.0f}")
        print(f"  Sisa kertas     : {mesin.get_stok_kertas()} lembar")
        print(f"  Sisa tinta      : {mesin.get_stok_tinta()} unit")
        print("="*55)
        mesin.cek_peringatan()
        return 'sukses'

def input_int(prompt: str, minimum: int = 0) -> int:
    while True:
        try:
            val = int(input(prompt))
            if val < minimum:
                print(f"  Nilai minimal {minimum}, coba lagi.")
            else:
                return val
        except ValueError:
            print("  Input harus angka bulat.")

def input_float(prompt: str, minimum: float = 0) -> float:
    while True:
        try:
            val = float(input(prompt))
            if val < minimum:
                print(f"  Nilai minimal {minimum}, coba lagi.")
            else:
                return val
        except ValueError:
            print("  Input harus angka.")

def tampilkan_status(mesin: MesinPrint):
    print("\n" + "-"*55)
```

```
    print("  STATUS MESIN")
    print(mesin.status_bar())
    print("-"*55 + "\n")

def lakukan_restock(mesin: MesinPrint):
    print("  (isi 0 jika tidak ingin restock item tersebut)")
    tambah_kertas = input_int("  Tambah Kertas (lembar) : ", 0)
    tambah_tinta  = input_int("  Tambah Tinta  (unit)   : ", 0)

    if tambah_kertas == 0 and tambah_tinta == 0:
        print("  Tidak ada yang di-restock.")
        if mesin.stok_habis():
            print("  >>> GAGAL: Stok masih kosong! Mesin tidak dapat digunakan
untuk print. <<<\n")
        else:
            print()
    else:
        mesin.restock(tambah_kertas, tambah_tinta)

    tampilkan_status(mesin)

def menu_setelah_transaksi(mesin: MesinPrint) -> str:
    while True:
        print("\n  1. Pelanggan berikutnya")
        print("  2. Restock kertas / tinta")
        print("  3. Lihat status mesin")
        print("  4. Keluar sistem")
        pilihan = input("\n  Pilih [1-4]: ").strip()
        if pilihan == "1":
            return 'lanjut'
        elif pilihan == "2":
            lakukan_restock(mesin)
            return 'lanjut'
        elif pilihan == "3":
            tampilkan_status(mesin)
        elif pilihan == "4":
            return 'keluar'
        else:
            print("  Pilihan tidak valid, coba lagi.")

def menu_stok_habis(mesin: MesinPrint) -> str:
    print("\n" + "!"*55)
    print("  STOK TIDAK CUKUP -- Harus restock sebelum lanjut!")
    print("!"*55)
    tampilkan_status(mesin)
    while True:
        print("  1. Restock sekarang")
        print("  2. Keluar sistem")
```

```
        pilihan = input("\n  Pilih [1/2]: ").strip()
        if pilihan == "1":
            lakukan_restock(mesin)
            if mesin.stok_habis():
                continue
            return 'lanjut'
        elif pilihan == "2":
            return 'keluar'
        else:
            print("  Pilihan tidak valid, coba lagi.")

if __name__ == "__main__":
    HARGA_PER_LEMBAR = 500.0

    print("\n" + "#"*55)
    print("   ITK-PRINT -- Sistem Layanan Fotokopi Mandiri")
    print("#"*55)

    print("\nSetup Mesin (sekali di awal)")
    print("-"*30)
    id_mesin    = input("  ID Mesin         : ")
    stok_kertas = input_int("  Stok Kertas awal : ", 0)
    stok_tinta  = input_int("  Stok Tinta awal  : ", 0)

    mesin = MesinPrint(id_mesin, stok_kertas, stok_tinta)
    tampilkan_status(mesin)

    while True:

        if mesin.stok_habis():
            aksi = menu_stok_habis(mesin)
            if aksi == 'keluar':
                break
            continue

        print("\nData Pelanggan Baru")
        print("-"*30)
        nama  = input("  Nama Pelanggan : ")
        saldo = input_float("  Saldo (Rp)     : ", 0)
        pelanggan = Pelanggan(nama, saldo)

        print("\nData Dokumen")
        print("-"*30)
        file = input("  Nama File      : ")
        hal  = input_int("  Jumlah Halaman : ", 1)
        dokumen = Dokumen(file, hal)
```

```
                hasil = pelanggan.cetak_dokumen(dokumen, mesin,
HARGA_PER_LEMBAR)

        if hasil in ('sukses', 'saldo_kurang'):
            aksi = menu_setelah_transaksi(mesin)
            if aksi == 'keluar':
                break

        elif hasil == 'stok_habis':
            aksi = menu_stok_habis(mesin)
            if aksi == 'keluar':
                break

    print(f"\n  Sistem ditutup.")
    print(f"  Total halaman tercetak : {mesin.get_total_cetak()} halaman")
    print("  Terima kasih!\n")
```

3.2 Penjelasan Alur dan Fungsi *Source Code*

Adapun penjelasan fungsi dari setiap bagian source code serta alur kerja Sistem Layanan Fotokopi Mandiri (ITK-Print) dapat dilihat pada tabel berikut.

Tabel 2. Fungsi *Source Code*

No	Nama Kelas / Fungsi	Fungsi	Penjelasan
1	`class Dokumen`	Representasi Dokumen	Menyimpan nama file dan jumlah halaman dokumen yang akan dicetak. Atribut jumlahHalaman bersifat privat dan hanya dapat diakses melalui method getJumlahHalaman().
2	`class MesinPrint`	Pengelola Mesin Cetak	Menyimpan data stok kertas dan tinta mesin. Seluruh atribut stok bersifat privat untuk mencegah modifikasi langsung dari luar kelas.
3	`cek_mesin_siap()`	Pengecekan Kesiapan Mesin	Memeriksa apakah stok kertas dan tinta keduanya bernilai lebih dari nol sebelum proses cetak diizinkan.
4	`stok_cukup_untuk (halaman)`	Validasi Stok	Memverifikasi apakah stok kertas dan tinta mencukupi

			sesuai jumlah halaman dokumen yang akan dicetak.
5	`kurangi_stok(halaman)`	Pengurangan Stok	Mengurangi stok kertas dan tinta secara proporsional sesuai jumlah halaman, serta mencatat total halaman yang telah dicetak.
6	`restock(tambah_kertas, tambah_tinta)`	Pengisian Ulang Stok	Menambah stok kertas dan/atau tinta dengan validasi agar tidak melebihi kapasitas maksimum yang telah ditetapkan.
7	`cek_peringatan()`	Notifikasi Stok Rendah	Menampilkan peringatan apabila stok kertas atau tinta telah berada di bawah ambang batas minimum (20 unit).
8	`status_bar()`	Tampilan Status Mesin	Menampilkan informasi visual berupa progress bar kapasitas kertas, tinta, dan total halaman yang telah dicetak.
9	`class Pelanggan`	Representasi Pelanggan	Menyimpan nama dan saldo pelanggan. Atribut saldo bersifat privat dan hanya dapat dimodifikasi melalui method potong_saldo().
10	`potong_saldo(jumlah)`	Pemotongan Saldo	Mengurangi saldo pelanggan sesuai biaya transaksi yang telah tervalidasi.
11	`cetak_dokumen(dokumen, mesin, harga)`	Proses Utama Transaksi	Memvalidasi ketersediaan stok mesin dan kecukupan saldo pelanggan sebelum proses cetak dijalankan. Jika gagal, sistem memberikan keterangan spesifik mengenai penyebab kegagalan.
12	`input_int() / input_float()`	Validasi Input	Memastikan bahwa input yang dimasukkan pengguna berupa angka valid dan tidak

			di bawah nilai minimum yang ditentukan.
13	`tampilkan_status(mesin)`	Tampilan Status	Menampilkan status terkini mesin cetak setelah setiap transaksi atau proses restock dilakukan.
14	`lakukan_restock(mesin)`	Proses Restock Interaktif	Menerima input jumlah kertas dan tinta yang akan ditambahkan, kemudian memanggil method restock() pada objek mesin.
15	`menu_setelah_transaksi(mesin)`	Menu Pasca-Transaksi	Menampilkan pilihan kepada pengguna setelah transaksi selesai, mencakup opsi pelanggan berikutnya, restock, lihat status mesin, atau keluar sistem.
16	`menu_stok_habis(mesin)`	Penanganan Stok Habis	Memaksa proses restock dilakukan sebelum sistem dapat melanjutkan ke transaksi berikutnya apabila stok mesin terdeteksi habis.

Tabel 3. Alur Kerja *Source Code*

Tahapan	Proses
1	Petugas melakukan setup mesin pada awal sesi dengan menginputkan ID mesin, stok kertas awal, dan stok tinta awal.
2	Sistem menampilkan status mesin beserta visual progress bar kapasitas kertas dan tinta.
3	Pelanggan memasukkan data diri (nama dan saldo) serta data dokumen yang akan dicetak (nama file dan jumlah halaman).
4	Sistem memvalidasi kecukupan stok mesin dan saldo pelanggan secara berurutan.
5	Apabila semua validasi terpenuhi, transaksi diproses: saldo pelanggan dipotong dan stok mesin dikurangi secara otomatis.
6	Jika stok mendekati batas minimum, sistem menampilkan notifikasi peringatan kepada pengguna.
7	Jika stok habis, sistem memaksa proses restock sebelum dapat melayani transaksi berikutnya.

3.3 Penjelasan Alur dan Fungsi *Source Code*

Pengujian program dilakukan melalui terminal Python untuk memverifikasi bahwa seluruh logika validasi dan aturan bisnis yang telah diimplementasikan berjalan sesuai dengan yang dirancang. Pengujian mencakup dua skenario utama, yaitu simulasi gagal dan simulasi sukses.

```
#######################################################
   ITK-PRINT -- Sistem Layanan Fotokopi Mandiri
#######################################################

Setup Mesin (sekali di awal)
-----------------------------
  ID Mesin         : 1
  Stok Kertas awal : 10
  Stok Tinta awal  : 10

-------------------------------------------------------
  STATUS MESIN
  Mesin       : 1
  Kertas      : [##............] 10/100
  Tinta       : [##............] 10/100
  Total cetak : 0 halaman
-------------------------------------------------------


Data Pelanggan Baru
-----------------------------
  Nama Pelanggan : valdi
  Saldo (Rp)     : 5000
```

Gambar 9. Pengisian Stok & Data Pelanggan

Pada skenario simulasi gagal, pengujian dilakukan terhadap tiga kondisi penolakan yang mungkin terjadi. Pertama, transaksi ditolak karena stok kertas dan/atau tinta pada mesin tidak mencukupi untuk memenuhi jumlah halaman yang diminta. Sistem secara otomatis menampilkan pesan kegagalan yang menginformasikan komponen mana yang kekurangan beserta jumlah stok yang tersisa. Kedua, transaksi ditolak karena saldo pelanggan lebih kecil dari total biaya yang harus dibayarkan, di mana sistem menampilkan rincian biaya yang dibutuhkan, saldo yang dimiliki, serta selisih kekurangan. Ketiga, pengisian ulang stok yang melebihi kapasitas maksimum mesin (100 unit) secara otomatis dibatasi oleh sistem hingga batas kapasitas yang diizinkan.

```
=====================================================
  TRANSAKSI -- valdi
-----------------------------------------------------
  File       : pbo
  Halaman    : 20 x Rp500 = Rp10,000
  Saldo      : Rp5,000
  Kertas     : 10 lembar | Tinta: 10 unit
-----------------------------------------------------
  GAGAL: Stok kertas (10) DAN tinta (10) tidak cukup untuk 20 halaman!
=====================================================

!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
  STOK TIDAK CUKUP -- Harus restock sebelum lanjut!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

-----------------------------------------------------
  STATUS MESIN
  Mesin       : 1
  Kertas      : [##............] 10/100
  Tinta       : [##............] 10/100
  Total cetak : 0 halaman
-----------------------------------------------------

  1. Restock sekarang
  2. Keluar sistem
```

Gambar 10. Skenario Gagal Print

Pada skenario simulasi sukses, pelanggan dengan saldo yang mencukupi dan mesin dengan stok yang memadai berhasil menyelesaikan transaksi pencetakan. Setelah transaksi berhasil, saldo pelanggan berkurang sesuai biaya yang telah dihitung, stok kertas dan tinta pada mesin berkurang secara proporsional sesuai jumlah halaman yang dicetak, dan total halaman tercetak pada mesin bertambah secara kumulatif. Hasil keluaran pada konsol memperlihatkan seluruh informasi transaksi secara lengkap dan terstruktur, termasuk detail biaya, sisa saldo, dan sisa stok mesin setelah transaksi selesai diproses.

```
=====================================================
  TRANSAKSI -- valdi
-----------------------------------------------------
  File       : pbo
  Halaman    : 20 x Rp500 = Rp10,000
  Saldo      : Rp30,000
  Kertas     : 30 lembar | Tinta: 30 unit
-----------------------------------------------------
  SUKSES: Dokumen dicetak!
  Saldo terpotong : Rp10,000
  Saldo tersisa   : Rp20,000
  Sisa kertas     : 10 lembar
  Sisa tinta      : 10 unit
=====================================================
  *** Kertas hampir habis! Sisa 10 lembar.
  *** Tinta hampir habis! Sisa 10 unit.

  1. Pelanggan berikutnya
  2. Restock kertas / tinta
  3. Lihat status mesin
  4. Keluar sistem
```

Gambar 11. Skenario Sukses Print

https://github.com/valdiitk/UTS-PBO